UNIVERSIDADE VEIGA DE ALMEIDA
CURSO DE ENGENHARIA DA COMPUTAÇÃO

LETÍCIA DOS REIS PRADO – 1240207303

TRABALHO:
Analisador Léxico da Linguagem DOMUS

Rio de Janeiro/RJ
2026

LETÍCIA DOS REIS PRADO

TRABALHO:
Analisador Léxico da Linguagem DOMUS

Trabalho para cumprimento do componente curricular Compiladores do Curso de Engenharia da Computação da Universidade Veiga de Almeida.

Orientador: Fábio Contarini Carneiro

Rio de Janeiro/RJ
2026

RESUMO

Este trabalho apresenta o desenvolvimento de um analisador léxico para a linguagem Domus, uma linguagem didática voltada à automação residencial, elaborado como parte da avaliação A1 da disciplina de Compiladores. O analisador foi implementado na linguagem C com o auxílio do gerador automático Flex, seguindo a organização em módulos do compilador TINY de Louden. O programa lê o código fonte de um arquivo externo e disponibiliza a função getToken(), que devolve a cada chamada o próximo token, exibindo de forma textual o token, seu atributo e a linha em que foi encontrado, até sinalizar o fim do arquivo.

Palavras-chave: Analisador léxico. Compiladores. Flex. Linguagem Domus. Tokens.

LISTA DE FIGURAS

FIGURA 1 – PROGRAMA DE TESTE PROG_TESTE.DMS 10
FIGURA 2 – SAÍDA DO ANALISADOR LÉXICO PARA O PROGRAMA DE TESTE P.1 11
FIGURA 3 – SAÍDA DO ANALISADOR LÉXICO PARA O PROGRAMA DE TESTE P.2 12
FIGURA 4 – SAÍDA DO ANALISADOR LÉXICO PARA O PROGRAMA DE TESTE P.3 13

LISTA DE TABELAS

TABELA 1 – LEXEMAS E TOKENS DA LINGUAGEM DOMUS 6
TABELA 2 – ARQUIVOS DO ANALISADOR LÉXICO 8

SUMÁRIO

1. INTRODUÇÃO 6
2. LEXEMAS E TOKENS 6
3. ESTRATÉGIA ADOTA PARA O DESENVOLVIMENTO DO Analisador Léxico 8
4. COMO FOI UTILIZAR O ANALISADOR LÉXICO AUTOMÁTICO 8
5. COMO COMPILAR E EXECUTAR O CÓDIGO 9
5.1. Requisitos: 9
5.2. Compilação 9
6. RESULTADO DA EXECUÇÃO DO PROGRAMA TESTE 10
REFERÊNCIAS 14

1. INTRODUÇÃO

A análise léxica é a primeira fase de um compilador. Nela, o programa fonte é lido caractere a caractere e agrupado em unidades significativas chamadas lexemas, que por sua vez são classificadas em tokens. O componente responsável por essa fase é o analisador léxico, também chamado de scanner ou programa de varredura. Além de reconhecer os tokens, o analisador léxico descarta espaços em branco e comentários e informa ao restante do compilador a posição (linha) em que cada token foi encontrado, o que é fundamental para a emissão de mensagens de erro compreensíveis.

2. LEXEMAS E TOKENS

Um lexema é a sequência de caracteres do programa fonte que casa com o padrão de um token, enquanto o token é a categoria abstrata à qual o lexema pertence.

Tabela 1 – Lexemas e tokens da linguagem Domus

Lexema	Token	Significado
sensor	SENSOR	Declaração de sensor (variável somente leitura)
atuador	ATUADOR	Declaração de atuador (variável somente escrita)
porta	PORTA	Define a porta do sensor/atuador
analogico	ANALOGICO	Dispositivo analógico (valores inteiros)
digital	DIGITAL	Dispositivo digital (valores 0 ou 1)
se	SE	Início da estrutura de decisão
entao	ENTAO	Bloco executado se a condição for verdadeira
senao	SENAO	Bloco executado se a condição for falsa
fim_se	FIM_SE	Final da estrutura de decisão

Lexema	Token	Significado
enquanto	ENQUANTO	Início da estrutura de repetição
fim_enquanto	FIM_ENQUANTO	Final da estrutura de repetição
ler	LER	Rotina de entrada de dados
escrever	ESCREVER	Rotina de saída de dados
+	MAIS	Operador de soma
-	MENOS	Operador de subtração
*	VEZES	Operador de multiplicação
/	DIV	Operador de divisão
<	ME	Menor que
<=	MEQ	Menor ou igual
>	MA	Maior que
>=	MAQ	Maior ou igual
==	IG	Igual
!=	DIF	Diferente
<-	ATRIB	Operador de atribuição
.	PONT	Separador de comandos
:	DPONT	Separador de tipo de sensor/atuador
,	VIRG	Separador de declarações
(	EPAREN	Abre bloco de expressão
)	DPAREN	Fecha bloco de expressão
sequência de dígitos (ex.: 30, 80)	NUM	Número inteiro; atributo = valor do lexema
letra seguida de letras/dígitos (ex.: cortina, asd)	ID	Identificador; atributo = nome do lexema
(fim do arquivo)	FIMARQ	Marca o fim do arquivo fonte
caractere não reconhecido (ex.: @, #)	ERROR	Erro léxico; atributo = lexema inválido

Fonte: Convencoes lexicas da linguagem Domus.pdf

3. ESTRATÉGIA ADOTA PARA O DESENVOLVIMENTO DO ANALISADOR LÉXICO

A estratégia adotada foi reaproveitar a arquitetura do analisador léxico da linguagem TINY estudado em aula (LOUDEN, 2004), adaptando-o para as convenções léxicas da linguagem Domus. Essa arquitetura separa o código em módulos com responsabilidades bem definidas, o que facilita a manutenção e a posterior integração com as fases seguintes do compilador (análise sintática e semântica).

Tabela 2 – Arquivos do analisador léxico

Arquivo	Conteúdo
global.h	Enumeração TokenType com todos os tokens, variáveis globais (source, listing, lineno) e flags de rastreamento.
scan.h	Interface do analisador: constante MAXTOKENLEN, vetor tokenString e o protótipo de getToken().
domus.l	Especificação léxica de entrada do Flex: expressões regulares, ações e a implementação de getToken().
util.h / util.c	Função printToken(), que exibe de forma textual cada token e seu atributo.
main.c	Programa de teste: abre o arquivo fonte, chama getToken() repetidamente até FIMARQ e encerra.

Fonte: Autoria

4. COMO FOI UTILIZAR O ANALISADOR LÉXICO AUTOMÁTICO

O desenvolvimento foi tranquilo. Como todos os tokens da linguagem puderam ser descritos apenas por expressões regulares, não foi necessário nenhum tratamento adicional na etapa de análise léxica, o que resultou em um scanner simples, eficiente e de rápida execução.

5. COMO COMPILAR E EXECUTAR O CÓDIGO

5.1. Requisitos:

Um compilador C (gcc, MinGW, clang ou equivalente) e o gerador Flex. Em ambiente Linux (Debian/Ubuntu) o Flex pode ser instalado com sudo apt install flex build-essential. Em Windows pode ser utilizado o pacote WinFlexBison (win_flex.exe) juntamente com o MinGW, ou o WSL. Nenhuma outra biblioteca externa é necessária.

5.2. Compilação

No diretório fonte execute:

flex domus.l

gcc -o domus_lex main.c util.c lex.yy.c

.\domus_lex prog_teste.dms

6. RESULTADO DA EXECUÇÃO DO PROGRAMA TESTE

Para validar o analisador utilizou-se o programa de teste ExemploDomus.txt fornecido junto ao trabalho, reproduzido na Figura 2. O programa exercita todos os tipos de declaração (sensor e atuador, analógico e digital), as estruturas se/senao e enquanto, atribuições, expressões aritméticas e relacionais, a rotina escrever e comentários de uma e de várias linhas.

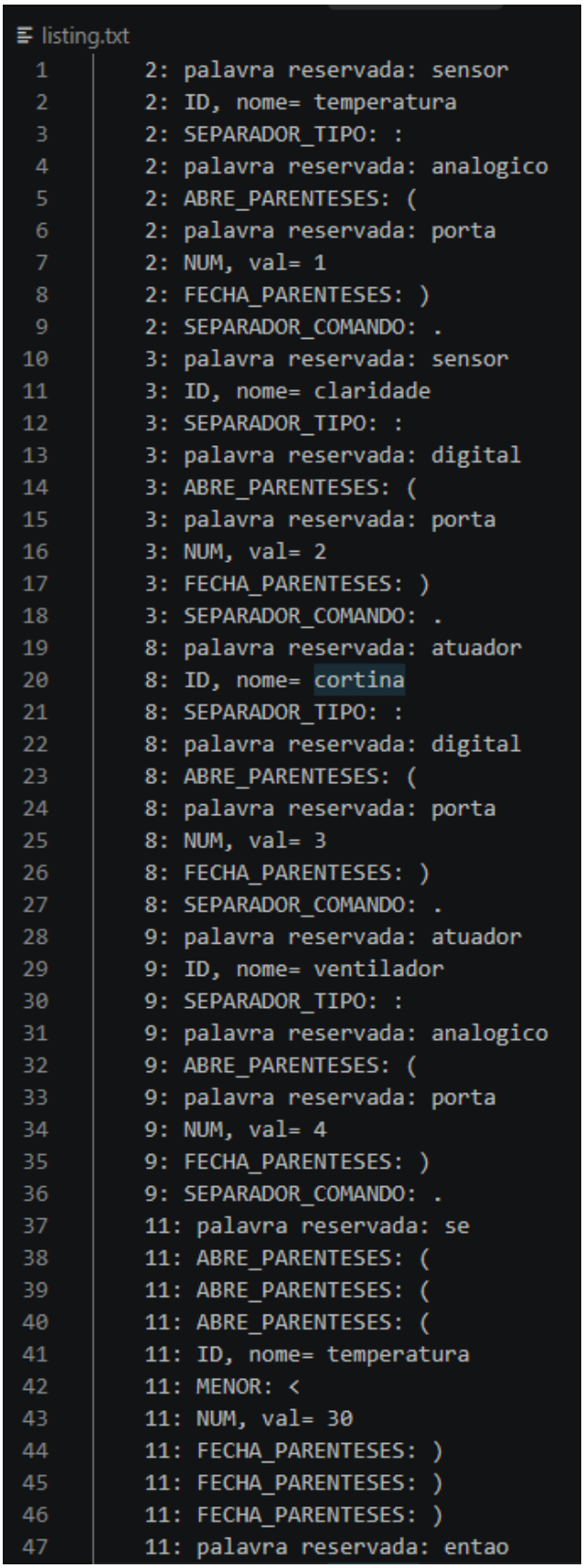
Figura 1 – Programa de teste prog_teste.dms

prog_teste.dms

```
1   /* Programa de teste */
2   sensor temperatura: analogico(porta 1).
3   sensor claridade:digital(porta 2).
4   /* multiplas
5   linhas
6   de comentario
7   */
8   atuador cortina: digital (porta 3).
9   atuador ventilador:analogico(porta 4).
10
11  se (((temperatura < 30))) entao
12      cortina <- 1.
13      ventiladora <- 80.
14  senao
15      cortina <- 0.
16      ventilador <- 0.
17  fim_se .
18  asd <- 8 .
19  asd <- asd + 2 .
20  escrever(asd+2) .
21  enquanto (asd>1)
22    asd <- asd -1.
23  fim_enquanto.
```

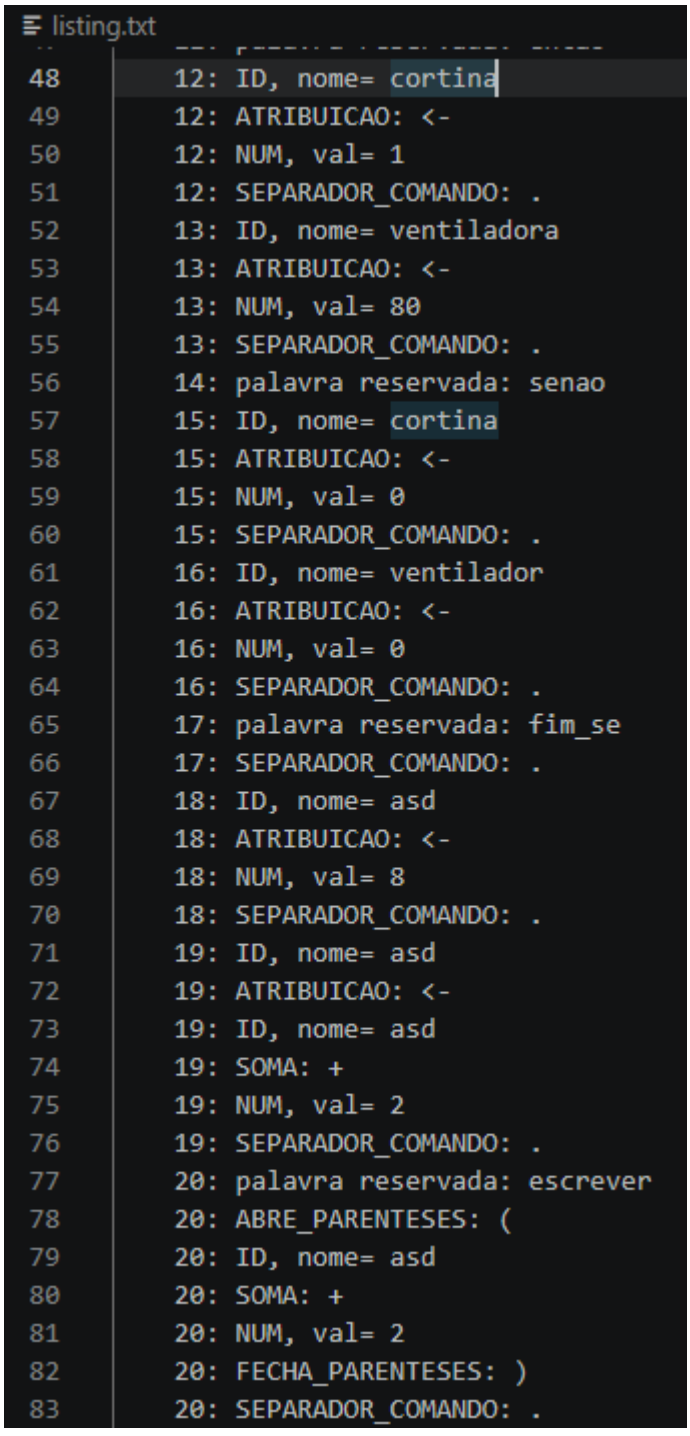
Fonte: Teams da Equipe

Figura 2 – Saída do analisador léxico para o programa de teste p.1



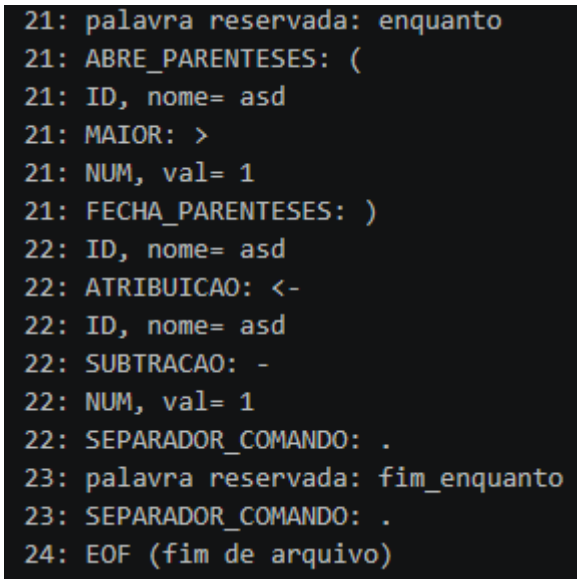
Fonte: Autoria

Figura 3 – Saída do analisador léxico para o programa de teste p.2



Fonte: Autoria

Figura 4 – Saída do analisador léxico para o programa de teste p.3



Fonte: Autoria

REFERÊNCIAS

Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. Compiladores: princípios, técnicas e ferramentas. 2ª ed. São Paulo: Pearson Addison-Wesley, 2008. Disponível na Biblioteca Virtual Pearson.

Louden, K. C. Compiladores: princípios e práticas. São Paulo: Centage Learning, 2004. Disponível na Minha Biblioteca.